

Exam 22 June 2023

① Regex that matches any odd nr. of words, each word having an odd nr. of letters.

$$^1 \left(\left([a-z][a-z] \right)^* [a-z] \right) \left(\left([a-z][a-z] \right)^* [a-z] \right) \left([a-z][a-z] \right)^* [a-z] \right)^* \$$$

word space word space word
 ↓ ↓ ↓ ↓ ↓
 one letter second letter final extra letter o/one/more times
 ↓
 repeat pairs
 (0, 2, 4, 6, ... letters) + 1
 odd length

② 4 commands that display the nr. of empty lines in a file.

1) `grep -c "$" filename`
 search lines count matches start of line end of line

nothing between start and end means empty line.

2) `grep -Ec "$" filename`
 extended regex (unnecessary) count runs once before file

print final sum.

3) `awk 'BEGIN {count=0} /^$/ {count++} END {print count}' filename`
 field-processing cmd. variable runs before reading the file nothing between start and end. runs after all lines are processed

4) `sed -n "/^$/p" filename | wc -l`
 don't print lines automatically search empty lines print those lines pipe (send output to next cmd) count only lines.

* 5) `grep -E -v -c ""`
 invert watch counts, not prints matches any character counts lines that contain NO characters.
 ↓
 the opposite of (x) character is ^character.

③ sed cmd that displays a file's lines deleting the first, third, fifth, ... space on those lines.

ex: Ana are multe / were foarte / bune.
 1 2 3 4 5
 space 1 ⇒ space 1-2

⇒ Ana are multe mere foarte bune.

sed -E 's/([^\]*) ([^\]*)/\1\2/g' filename
 capture group any char except space 0 or more group 1 space ✗ space ✓ group 2 inserts whatever text matched inside the group 1 repeat this replacement for every pair of words it finds on the line "1-1" - group 2

④ awk cmd that displays the product of the last field of lines on odd pos. having an odd nr. of fields.

→ take lines on odd positions → count their fields → take only those lines with an odd nr. of fields → take their last fields of each → multiply them.

awk 'BEGIN {P=1} NR%2==1 && NF%2==1 {P*=\$NF} END {print P}' filename
 runs before reading the file. product variable (nr. row) line number (current) (nr. field) nr. of fields on current line last field

⑤ 4 ways of redirecting the stdout of a process.

1) command > file
 ↓ ↓ destination
 program redirect (overwrite)

2) command >> file
 ↓
 append stdout

3) command | another command
 stdout stdin

4) command > file 2>&1
 stderr redirect same place as stdout
 file descriptor

Others:

command 1> file
 command | tee file

{ stdout → screen simultaneously
 { stdout → file

3 SHELL conditions that check the $\exists$ of a file.

① 1) if test ^{exists} -e ^{filename} file; then
echo "Exists"
fi

2) if ^{with spaces!} [-e file]; then
echo ...
fi

3) if [[-e file]]; then
echo ...
fi
→ safer
→ Bash's extended test
→ supports more operations

④ Hierarchy of a process (incl. parent)

```
for i=0; i<3; i++) {  
    if (execvp("ls", "ls", "/", NULL) != -1) {  
        fork();  
    }  
}
```

→ success (P)

p is replaced by ls through execvp; no children are created.

⑧ Add code s.t. the instruction does not get stuck waiting for stdin.

```
exec execvp("cat", NULL);
```

If started without files, cat reads from stdin (keyboard, usually) ⇒ cat waits ∞.

1) ⇒ execvp("cat", "cat", "file.txt", NULL);

2) redirected stdin.
→ provide file
→ special empty file
→ open for reading

source descriptor →
int fd = open("/dev/null", O_RDONLY);
dup2(fd, STDIN_FILENO);
close(fd);
execvp("cat", "cat", NULL); ← cat reads from /dev/null
descriptor 0 (stdin becomes /dev/null)

⑨ Implementation of popen, pclose functions, only for the case when the command output should be read in the C code.

POpen:
- create a pipe
- fork process
- child:
 - close read end of the pipe,
 - redirect stdout to pipe
 - execute command with execvp
- parent:
 - close write end of pipe
 - return file pointer for reading

PClose:
- close file pointer
- wait for child process
- return child exit status

⑩ How many FIFOs can a process open for reading if the FIFOs will be used by other processes only for reading?

no body writes

`open("a2b", O_RDONLY);`

↳ blocks until some process opens FIFO for writing, but nobody writes $\Rightarrow$ block ∞

$\Rightarrow$ ~~0 processes open~~ 0 FIFOs opened.

(normal) ...
`(mkfifo("a2b", 0600);
mkfifo("b2a", 0600);
int a2b = open("a2b", O_WRONLY);
int b2a = open("b2a", O_RDONLY);`

! * for regular files $\Rightarrow$ depends on `ulimit -m` (see 29 jun 2023, ex. 10)

⑪ When FIFO over PIPE?

- when unrelated processes need to communicate.
- FIFOs don't require a common ancestor process.
- FIFOs have a pathname in the system
- FIFOs persist after processes terminate, while PIPES only exist when their processes exist (the process that uses a pipe dies $\Rightarrow$ pipe dies).

⑫ Critical section

$\Rightarrow$ Section in the program where shared resources (such as memory var.) are accessed and maybe modified. $\Rightarrow$ prevent stability using mutual exclusion.

⑬ When MUTEX over RWLOCK?

`pthread_mutex_lock(&m)` $\rightarrow$ one thread enters only.

`pthread_rwlock_rdlock(&rw)` $\rightarrow$ more readers can enter.

`pthread_rwlock_wrlock(&rw)` $\rightarrow$ one writer enters only.

$\Rightarrow$ when there are many write operations.

$\Rightarrow$ Mutex is better when there are many write operations / few readers (few readers and many writers)

⑭ Replacing calls to `pthread_mutex_lock` with calls to sem_wait

mutex $\Rightarrow$ 1 thread

sem $\Rightarrow$ n threads.

If sem is initialized with value 1 $\Rightarrow$ same as mutex.

- value $> 1 \Rightarrow$ more threads may enter the critical section simultaneously $\Rightarrow$ race conditions.

Mutual exclusion is preserved $\Leftrightarrow$ semaphore initialised with value 1.

Producer - consumer.

Shared buffer: $\text{buffer}[N] \Rightarrow \text{mutex}$ protects buffer

producer: adds items

consumer: removes items

prevent - producer writing into full buffer $\Rightarrow$ full semaphore

- consumer reading from empty buffer $\Rightarrow$ empty semaphore

$m = \text{buffer size}$

$p = \text{producer position}$

$c = \text{consumer position}$

Full buffer: $p == m$ $(p+1) \% m == c$

Empty buffer: $p == c$

↓

Producer:

empty --

full ++

↓

$\text{sem_wait}(\text{empty})$

$\text{mutex_lock}(\text{mutex})$

insert item

$\text{mutex_unlock}(\text{mutex})$

$\text{sem_post}(\text{full})$

Consumer:

full --

empty ++

↓

$\text{sem_wait}(\text{full})$

$\text{mutex_lock}(\text{mutex})$

remove item

$\text{mutex_unlock}(\text{mutex})$

$\text{sem_post}(\text{empty})$

⑪ Prevent deadlocks: (+ see 19 June '23)

- consistent locking order

- avoid nested locks

? - deadlock detection algorithm

? - lock timeouts

⑫ What state transition will a process undergo when calls pthread_cond_wait?

$\Rightarrow$ release mutex $\rightarrow$ block thread $\rightarrow$ wait for signal $\rightarrow$ reacquire signal $\rightarrow$ continue

Running state $\Rightarrow$ blocked state (waiting) $\Rightarrow$ ready state $\Rightarrow$ running state.

When pthread_cond_wait is called, the process releases the associated mutex and blocks itself; it remains blocked until another process signals the condition with pthread_cond_signal; after condition is signalled, process is awakened, it reacquires the mutex and gets back to running state.

(19) Content of file of type DIRECTORY (-d) in the Linux file system?

! Directory does not contain files only. It contains (filename $\rightarrow$ inode number) associations.

Directory contains:

- 1) directory entries (file names, inode no.)
- 2) $\cdot \rightarrow$ inode of current directory
- 3) $\cdot\cdot \rightarrow$ inode of parent directory
- 4) file1 $\rightarrow$ inode 123
- 5) file2 $\rightarrow$ inode 456
- $\vdots$

(20) Symbolic vs Hard Links.

Symbolic Link

- points to the path of the target file
- can span across $\neq$ file systems
- invalid if target file is deleted
- created with `ln -s`

Hard Link

- | | | | |
|---|---|---|--|
| { | hard link $\rightarrow$ inode
inode $\in$ one filesystem
$\neq$ partition $\Rightarrow \neq$ filesystem | } | • <u>cannot</u> span
across $\neq$
filesystems |
|---|---|---|--|
- points to the inode of the target file
 - valid as long as $\exists$ at least one link to the inode
 - created with `ln`

② 4 GREP cmd. that display lines that contain words that begin with uppercase / lowercase; (file has words separated by spaces)

1) `grep -E ' (^| ) [aeiouAEIOU] [a-zA-Z]* ($| ) ' filename`

Annotations:
 - `(^| )`: start of line OR space
 - `[aeiouAEIOU]`: first is vowel
 - `[a-zA-Z]*`: anything else after
 - `($| )`: end of line or space.

2) `grep -E '\<[aeiouAEIOU][a-zA-Z]*\>' filename`

Annotations:
 - `\<`: start of word
 - `\>`: end of word

3) `grep -E -i ' (^| ) [aeiouAEIOU] [a-zA-Z]* ($| ) ' filename`

Annotations:
 - `-i`: ignore case

4) `grep -Ei '\<[aeiou][a-zA-Z]*\>' filename`

Annotations:
 - `-Ei`: ignore case

2 SED commands → lines that don't contain any digits (file has letters / digits sep. by spaces).

1) `sed -m '/[0-9]/!p' filename`

Annotations:
 - `-m`: thing gets printed unless specified with p
 - `/[0-9]/`: search digits
 - `!p`: print lines that DON'T match the previous condition.

2) `sed -m '/^[^0-9]*$/p' filename`

Annotations:
 - `^`: start of line
 - `[^0-9]*`: 0 or more non-digits
 - `$`: end of line
 - `/p`: print lines that DO match the previous condition.